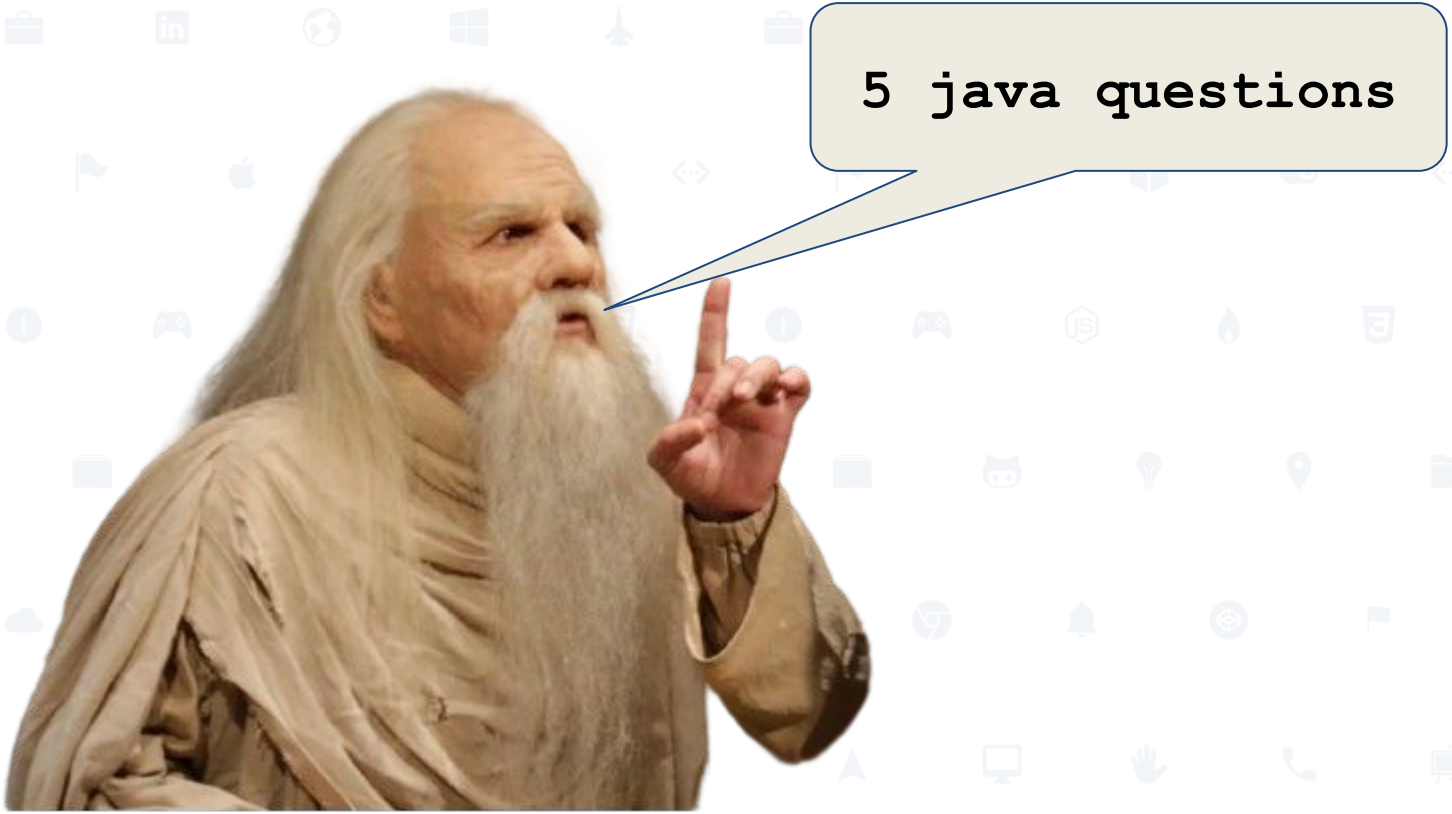
Lesson 28

02.04.2026

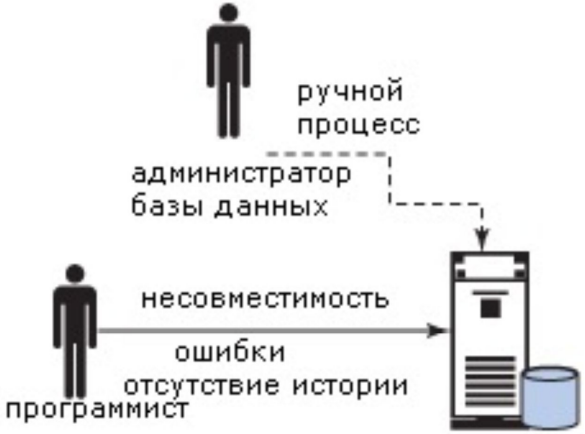

5 java questions

1. Що станеться, якщо в HashMap додати елемент із уже існуючим ключем?
2. Як обчислюється індекс кошика (bucket) у HashMap для заданого ключа?
3. Чи можна використовувати null як ключ або значення в HashMap?
4. Що означає параметр **load factor** у HashMap і чому він дорівнює за замовчуванням?
5. Що відбувається, якщо кількість елементів в одному кошику перевищує певний поріг?

Міграція баз даних

Розробка програмного забезпечення призводить до таких проблем:

- ручне внесення змін в БД;
- різні версії БД у різних учасників команди розробників;
- непослідовні підходи до внесення змін (в базу даних або дані);
- неефективні механізми ручного управління змінами при переходах між версіями баз даних.



Flyway

Управління змінами в базі даних за допомогою LiquiBase (www.liquibase.rg)

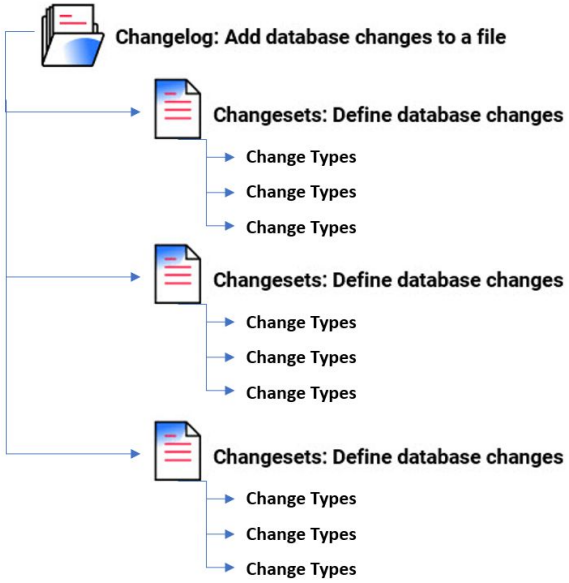
Liquibase — це незалежна від бази даних бібліотека для відстеження, керування та застосування змін схеми бази даних. Для того, щоб внести зміни в БД, створюється файл міграції (*changeset*), який включається в головний файл (*changeLg*), який контролює версії та керує всіма змінами. У якості опису структури та змін бази даних використовуються формати XML, YAML, JSN і SQL.

Основні функціональні можливості:

- Оновлення бази даних до поточної версії
- Відкат останніх змін у базі даних / на визначену дату / час / тега
- SQL для оновлення бази даних і відкатів можна зберегти для ручного перегляду
- «Контексти» для включення / виключення набору змін для виконання
- Отчет о различиях баз данных
- Генерація змін в базі даних
- Можливість створення журналу змін для створення існуючої бази даних
- Створення документації по змінам бази даних
- Перевірка СУБД, перевірка користувача та попередні умови перевірки SQL
- Можливість розбивати журнал змін на кілька файлів для посилення управління
- Виконується через командну строку, Apache Ant, Apache Maven, контейнер сервлетів або Spring Framewrk

Щоб розпочати роботу з LiquiBase, потрібно виконати чотири кроки:

- Створити файл із журналом змін у базі даних (**change log**).
- визначити набір змін (**change set**) у цьому файлі.
- Призначити набір змін до бази даних через командну строку або сценарій збірки.
- Перевірити зміни в базі даних.



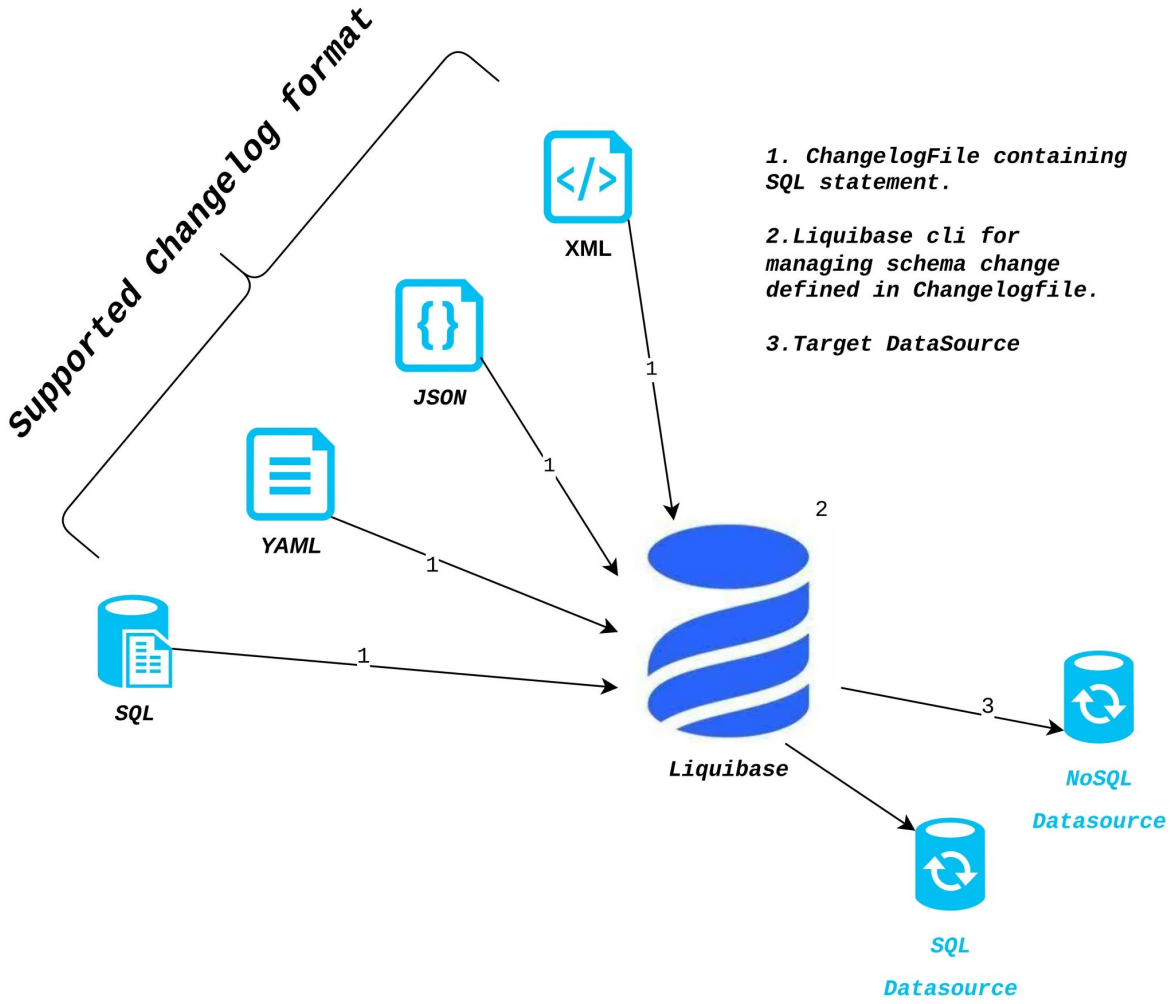
Changesets: Define database changes

Changelog: Add database changes to a file

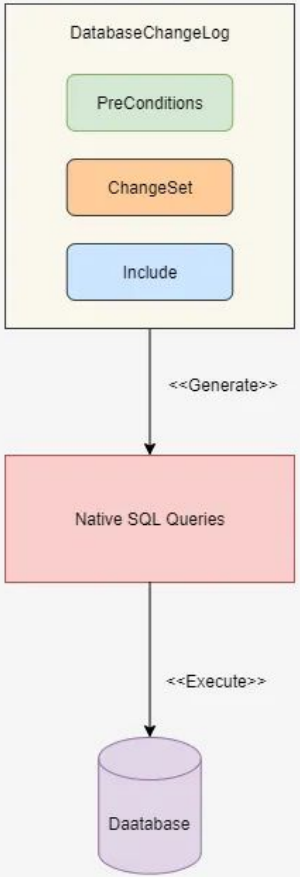
Update: Run the command to deploy database changes

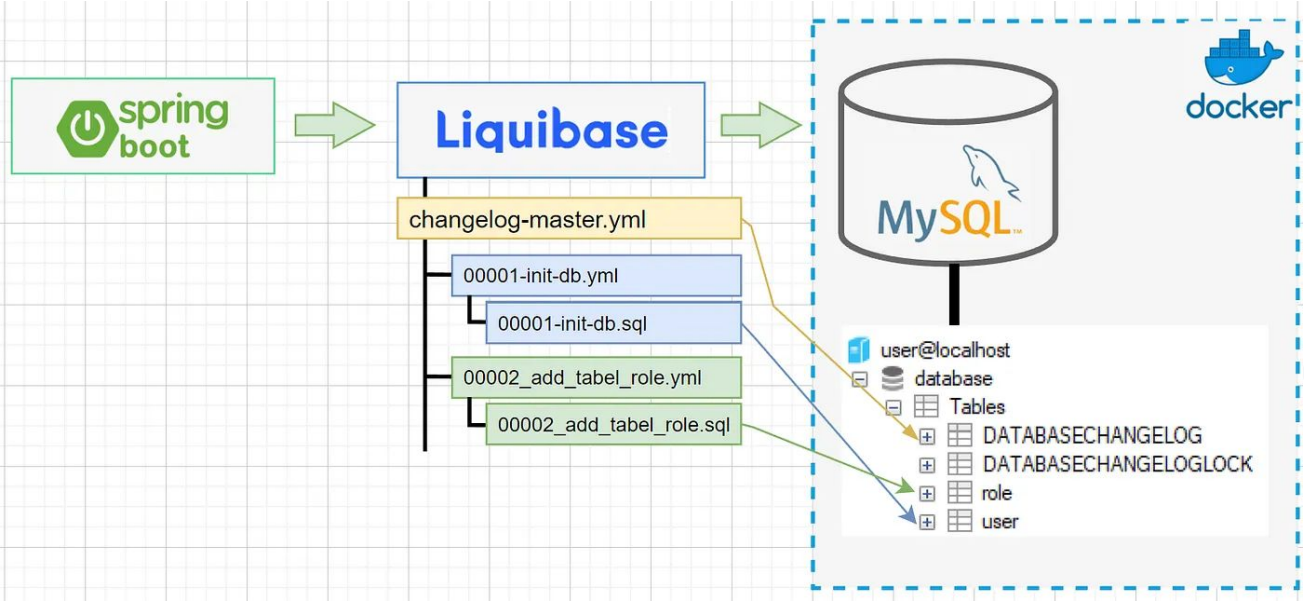
DATABASECHANGELOG and DATABASECHANGELOGLOCK:
Track and version database changes

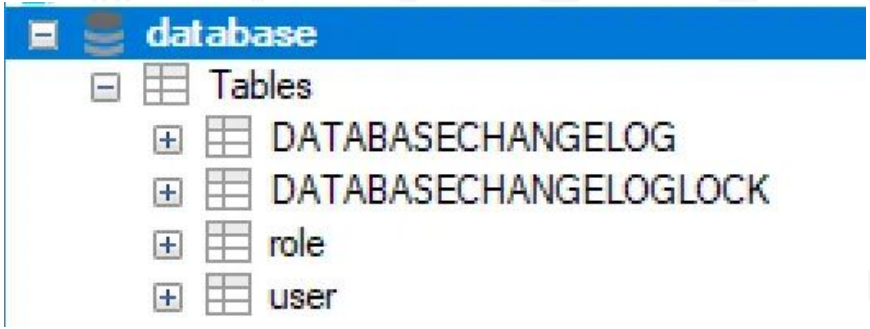
Manage your database changes with other Liquibase commands

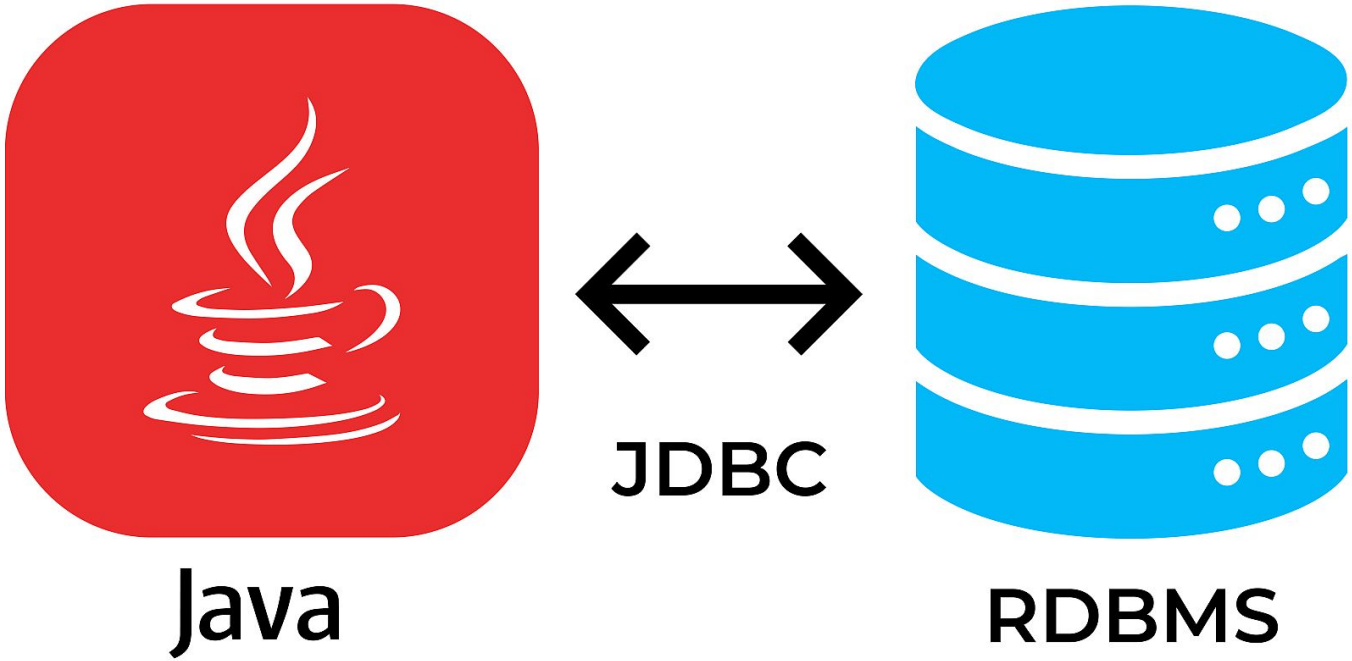

Supported Changelog format
XML
JSON
YAML
SQL
1
1
1
1
2
3
Liquibase
NoSQL
Datasource
SQL
Datasource
1. ChangelogFile containing SQL statement.
2.Liquibase cli for managing schema change defined in Changelogfile.
3.Target DataSource

How Liquibase Works

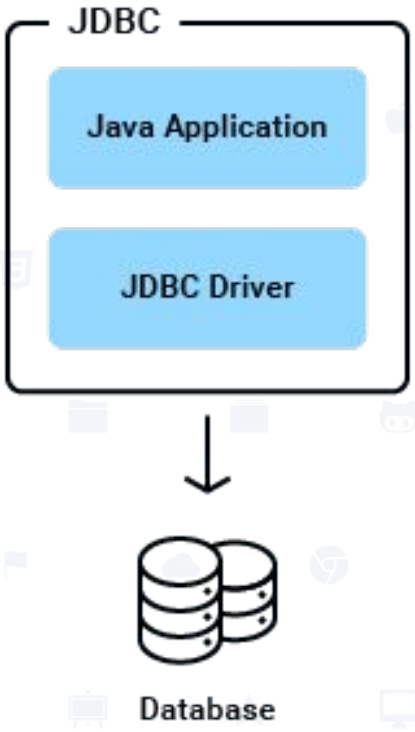



spring boot
Liquibase
changelog-master.yml
00001-init-db.yml
00001-init-db.sql
00002_add_tabel_role.yml
00002_add_tabel_role.sql
docker
MySQL
user@localhost
database
Tables
DATABASECHANGELOG
DATABASECHANGELOGLOCK
role
user


database
Tables
DATABASECHANGELOG
DATABASECHANGELOGLOCK
role
user

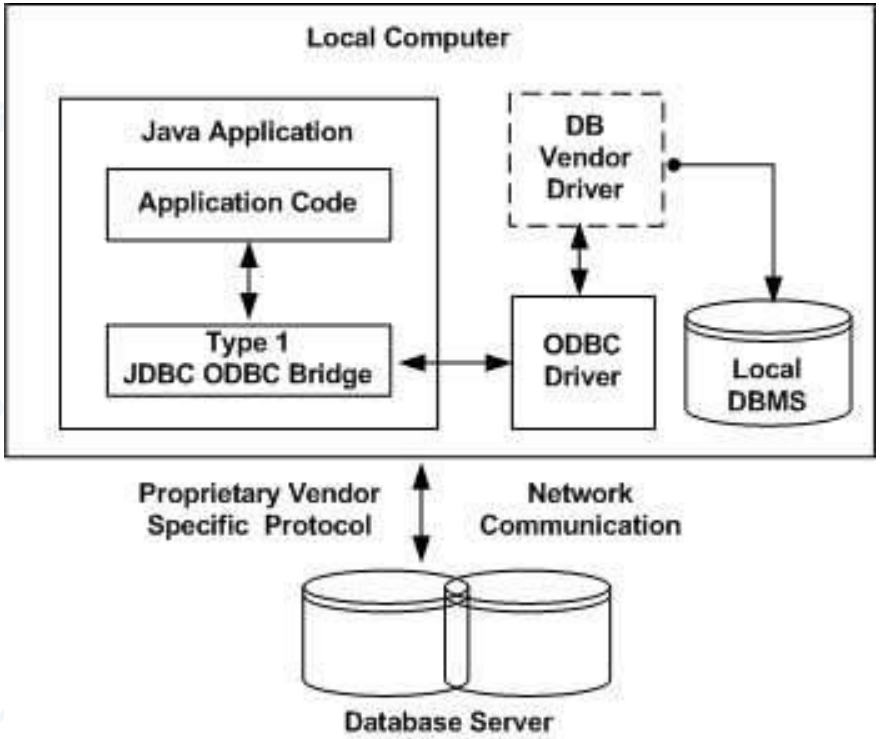

Java
JDBC
RDBMS

JDBC (Java DataBase Connectivity - з'єднання з базами даних на Java) призначений для взаємодії Java-додатки з різними системами управління базами даних (СУБД). Весь рух в JDBC засновано на драйверах, які вказуються спеціально описаним URL.



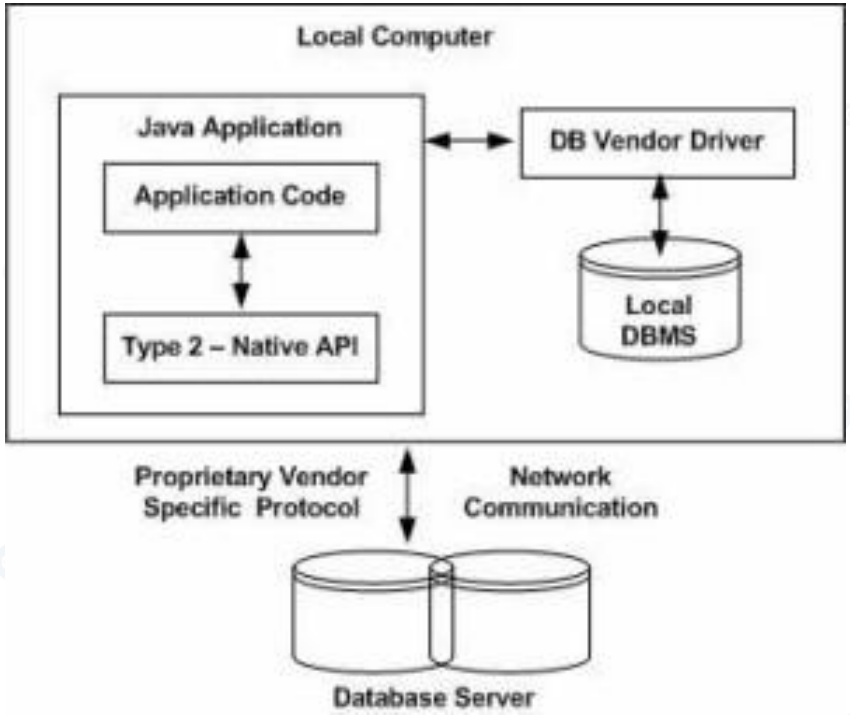
JDBC – ODBC транслятор

Цей тип драйвера транслює JDBC у встановлений на кожній машині клієнтську машину ODBC. Використання ODBC вимагає конфігурації DSN, який є цільовою базою даних.



JDBC – нативний API

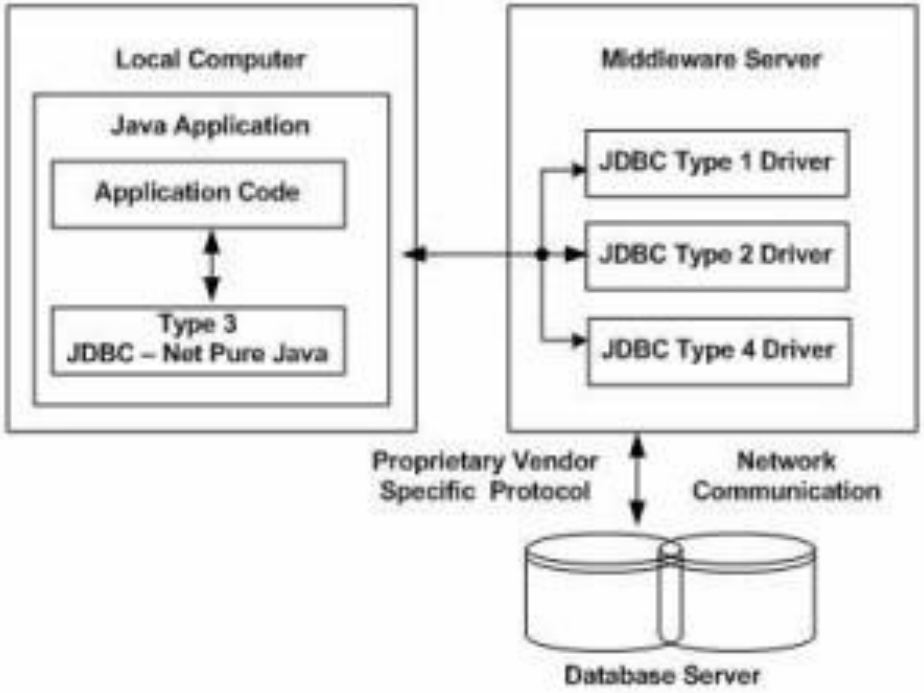
У цьому драйвері JDBC API перетворюється на унікальний кожної БД нативний C/C++ API. Його принцип роботи вкрай схожий на драйвер першого типу.



JDBC драйвер на основі бібліотеки Java

Цей тип драйверів використовує триланковий підхід для отримання доступу до БД. Для зв'язку з проміжним сервером програми використовується стандартний мережевий сокет. Інформація, отримана від цього сокету, транслюється проміжним сервером у формат, який необхідний для конкретної БД і направляється в сервер БД.

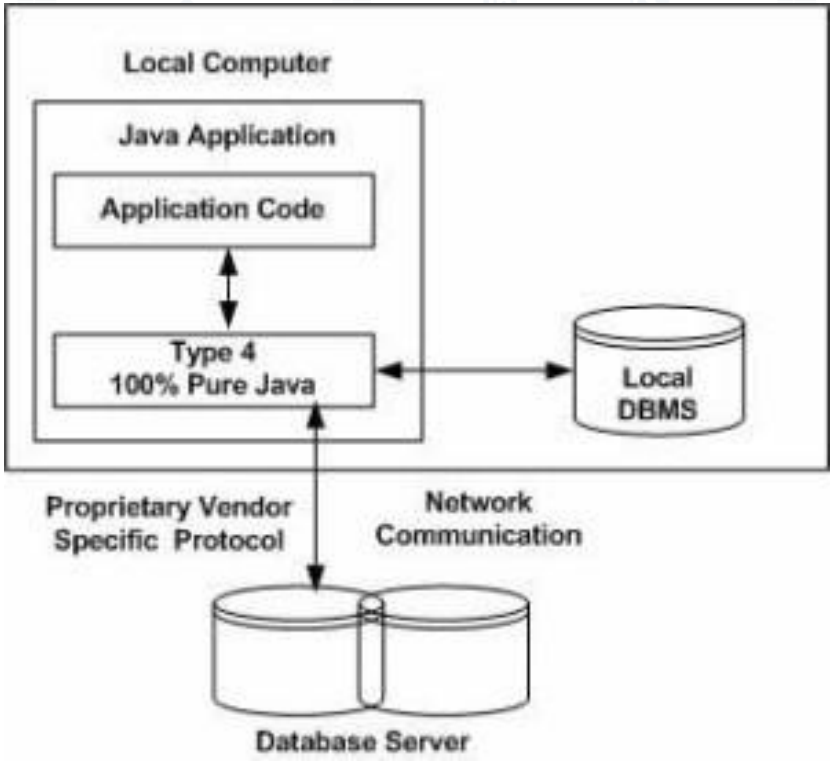
.



Чиста Java

Цей тип драйверів розроблений повністю з використанням мови програмування Java та працює з БД через сокетне з'єднання. Головна його перевага - найбільша продуктивність і, як правило, надається розробником БД.

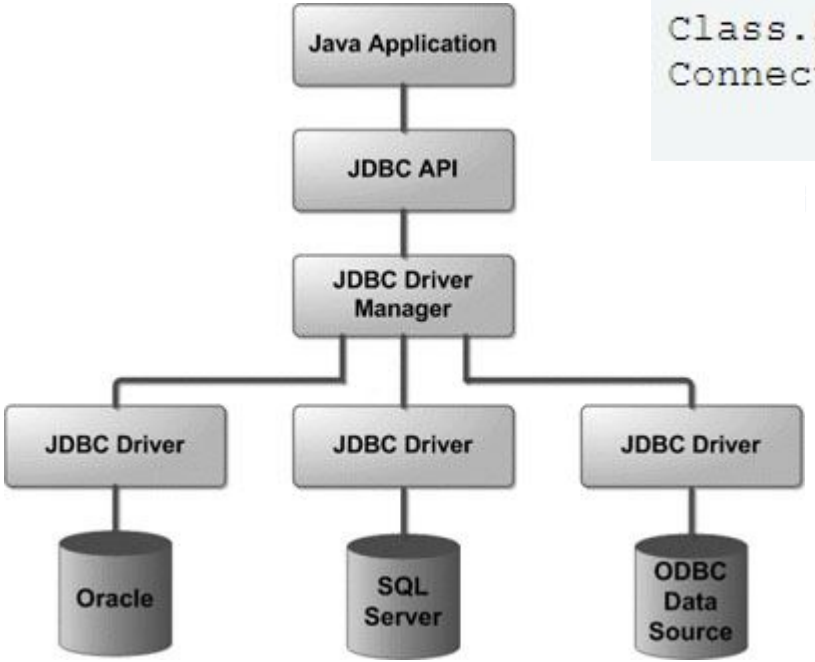
.



```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
</dependency>
```

```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.6.0</version>
</dependency>
```

```
<dependency>
    <groupId>com.oracle.database.jdbc</groupId>
    <artifactId>ojdbc8</artifactId>
    <version>23.2.0.0</version>
</dependency>
```

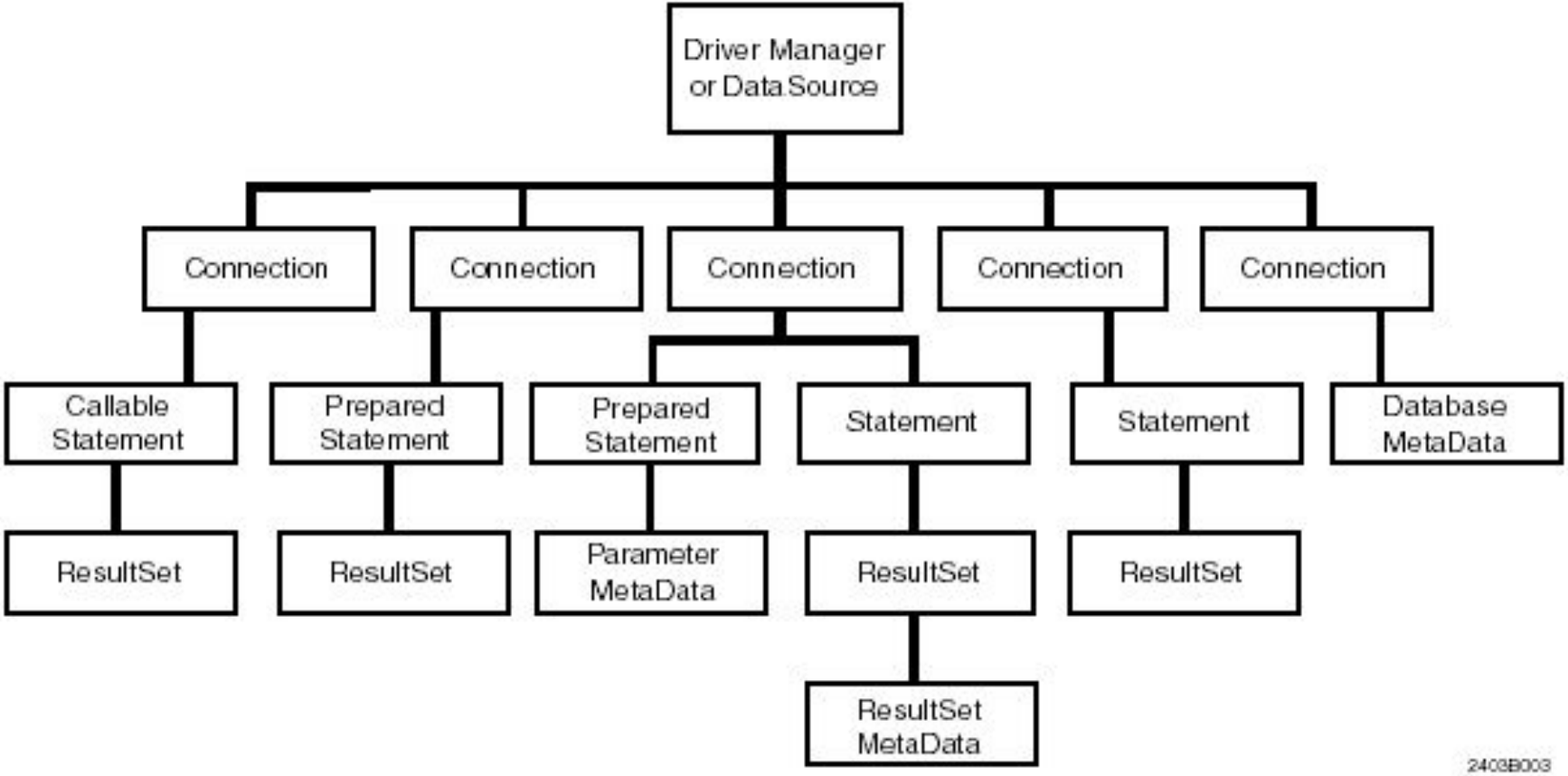


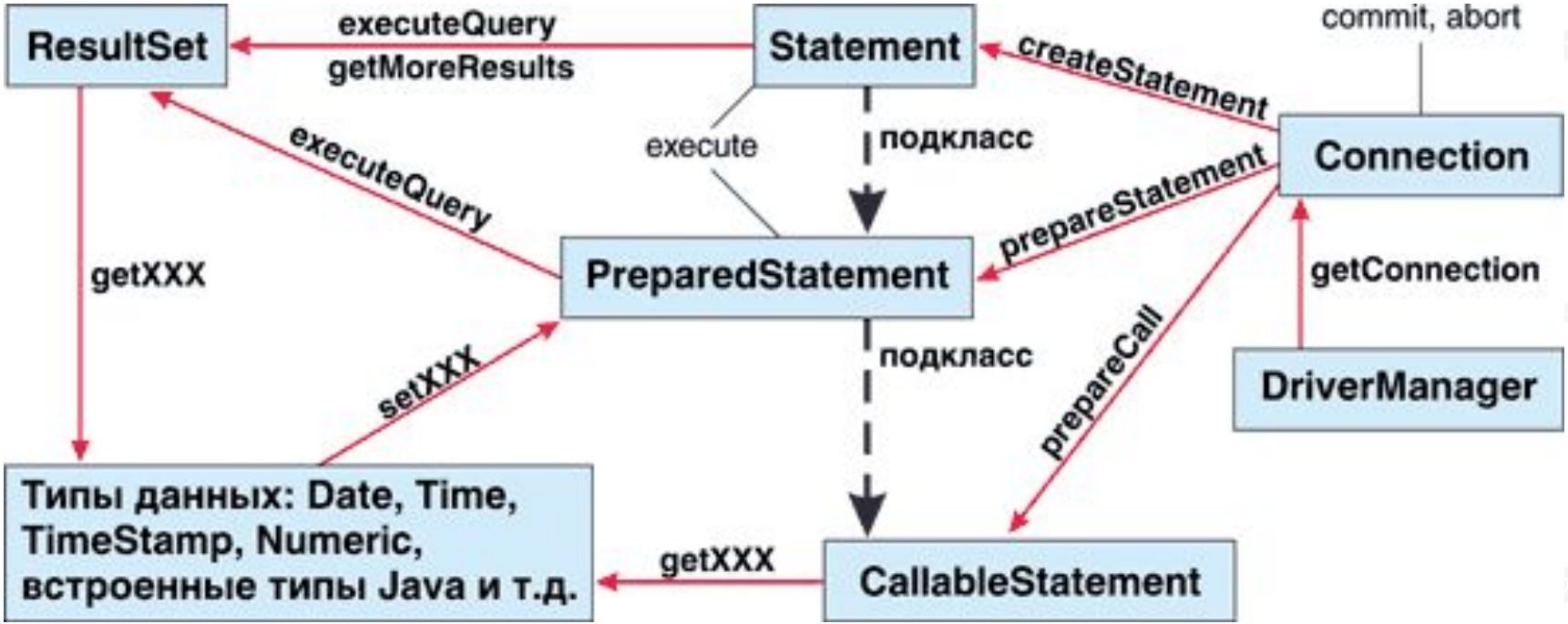

```
Class.forName(driverClass);
Connection connection = DriverManager
        .getConnection(url, user, password) ;
```

Де **driverClass** - це рядок із повним ім'ям класу JDBC драйвера, наприклад org.h2.Driver для H2 Database або com.mysql.jdbc.Driver для MySql.

DriverManager - це синглтон, який містить інформацію про всі зареєстровані драйвери. Метод getConnection на основі параметра URL знаходить java.sql.Driver відповідної бази даних і викликає метод connect.

```
Class.forName("com.mysql.jdbc.Driver");
Connection connect = DriverManager
        .getConnection( url: "jdbc:mysql://localhost:3306/student?"
                + "user=root&password=root");
```


Driver Manager or Data Source
Connection
Connection
Connection
Connection
Connection
Callable Statement
Prepared Statement
Prepared Statement
Statement
Statement
Database MetaData
ResultSet
ResultSet
Parameter MetaData
ResultSet
ResultSet
ResultSet MetaData
2403B003


ResultSet
executeQuery
getMoreResults
Statement
commit, abort
createStatement
execute
подкласс
Connection
executeQuery
prepareStatement
getXXX
PreparedStatement
getConnection
setXXX
подкласс
prepareCall
DriverManager
Типы данных: Date, Time, TimeStamp, Numeric, встроенные типы Java и т.д.
getXXX
CallableStatement

```java
Statement statement = connect.createStatement();
statement.executeQuery( sql: "select * from city");
```

```java
PreparedStatement preparedStatement = connection.prepareStatement( sql: "insert into city(city) values (?)");

List<String> cityList = Arrays.asList("London", "Paris", "Madrid", "Berlin");

cityList.forEach(city -> {
    try {
        preparedStatement.setString( parameterIndex: 1, city);
        preparedStatement.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    }
});
```

DatabaseMetaData

С помощью Connection можно получить очень полезную сущность DatabaseMetaData. Она позволяет получить метаинформацию о схеме базы данных, а именно какие в базе данных есть объекты - таблицы, колонки, индексы, триггеры, процедуры и так далее.